



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL

INTERDISCIPLINARIA EN INGENIERIA Y TECNOLOGIAS
AVANZADAS



BASES DE DATOS DISTRIBUIDAS

PRÁCTICA 4:
Problemas de Concurrencia

INTEGRANTES:

MIRAMAR CARDOSO ALEYDI

REYES JIMENEZ ADAEL

RODRÍGUEZ JIMÉNEZ OCTAVIO ALEJANDRO

UGALDE ESCOBAR DONALDO PATRICIO

PROFESOR: CARLOS DE LA CRUZ SOSA

2TV3

INTRODUCCIÓN

La gestión de transacciones y el control de concurrencia son pilares fundamentales en el diseño y administración de bases de datos relacionales y distribuidas. Cuando múltiples usuarios o procesos intentan acceder y modificar los mismos datos de manera simultánea, pueden presentarse fenómenos anómalos de lectura, tales como las lecturas sucias, las lecturas no repetibles y las lecturas fantasmas. Para mitigar estos problemas y preservar la integridad de la información, los sistemas gestores de bases de datos (SGBD) operan mediante transacciones regidas por las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) y aplican distintos niveles de aislamiento que equilibran la consistencia rigurosa con el rendimiento del sistema. El presente análisis documenta la simulación técnica de estas anomalías de concurrencia utilizando SQL Server y la base de datos AdventureWorks, sirviendo como fundamento teórico y práctico para abordar procesos arquitectónicos más complejos, como la fragmentación de tablas y el mantenimiento de la integridad referencial en entornos distribuidos.

LECTURA SUCIA

Una lectura sucia en bases de datos ocurre cuando una transacción lee información que está siendo modificada por otra operación, pero que todavía no ha sido guardada de forma definitiva o confirmada. Básicamente, estamos leyendo datos temporales que podrían cambiar o incluso cancelarse por completo.

Ejemplo:

Proceso A:

```
---PROCESO A: LECTURA SUCIA -----  
USE AdventureWorks2025;  
GO  
  
-- Iniciamos una transacción abierta  
BEGIN TRANSACTION;  
  
-- Valor original  
DECLARE @PrecioOrigen MONEY;  
  
select @PrecioOrigen = ListPrice  
      from Production.Product  
     where ProductID = 706;  
PRINT 'Proceso A: Precio del producto original:' + CAST(@PrecioOrigen AS VARCHAR(20));  
  
-- Modificamos el precio de un producto como en el ejemplo del libro  
UPDATE Production.Product  
SET ListPrice = ListPrice * 1.2  
WHERE ProductID = 706;  
  
PRINT 'Proceso A: Precio del producto actualizado.  
      Esperando 15 segundos antes de hacer ROLLBACK...';  
  
-- Pausamos la ejecución por 15 segundos para ejecutar el Proceso B  
WAITFOR DELAY '00:00:15';  
  
-- Simulamos la cancelación, revirtiendo el cambio de precio  
ROLLBACK TRANSACTION;  
PRINT 'Proceso A: Transacción revertida (ROLLBACK ejecutado).  
      El precio original se ha restaurado.';  
GO
```

Proceso B:

```
---PROCESO B: LECTURA SUCIA -----  
USE AdventureWorks2025;  
GO  
  
-- Config el nivel de aislamiento para permitir lecturas sucias  
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
  
PRINT 'Proceso B: Leyendo datos no confirmados...';
```

```
DECLARE @PrecioInvalido MONEY;

-- Consulta del precio que el Proceso A modificó pero aún no confirma
SELECT @PrecioInvalido = ListPrice
FROM Production.Product
WHERE ProductID = 706;

PRINT 'Proceso B: El precio sucio leído es: ' + CAST(@PrecioInvalido AS VARCHAR(20));

-- Simulación de la consecuencia
PRINT 'Proceso B: ATENCIÓN - Si insertamos un registro en Sales.SalesOrderDetail ahora,
      usariamos el precio incorrecto (Precio = ' + CAST(@PrecioInvalido AS VARCHAR(20)) + ').';

GO
```

Resultado en Proceso A:

```
Proceso A: Precio del producto original:1431.50

(1 fila afectada)
Proceso A: Precio del producto actualizado.
      Esperando 15 segundos antes de hacer ROLLBACK...
Proceso A: Transacción revertida (ROLLBACK ejecutado).
      El precio original se ha restaurado.

Hora de finalización: 2026-09-22T15:27:28.2352453-06:00
```

Resultado en Proceso B:

```
Proceso B: Leyendo datos no confirmados...
Proceso B: El precio sucio leído es: 1717.80
Proceso B: ATENCIÓN - Si insertamos un registro en Sales.SalesOrderDetail ahora,
      usariamos el precio incorrecto (Precio = 1717.80).

Hora de finalización: 2026-09-22T15:27:20.1343729-06:00
```

Conclusión:

Los resultados que vemos en la imagen muestran claramente el enorme riesgo de este problema de concurrencia. El Proceso B terminó capturando un precio sucio de 1717.80, asumiendo que era un valor real y válido. La gravedad de lo que obtuvimos es que, si el Proceso B hubiera continuado su camino para registrar una venta real, habría utilizado esos 1717.80 para cobrarle al cliente. Al final, el Proceso A revirtió todos sus cambios y el precio volvió a su estado original, dejando al Proceso B trabajando con información falsa. Esto nos demuestra que sacrificar el aislamiento de las transacciones puede arruinar por completo la integridad de los datos en un sistema.

LECTURA NO REPETIBLE

Una lectura no replicable ocurre cuando una misma transacción lee un registro o un conjunto de datos varias veces seguidas, pero justo en el intermedio, otra operación diferente entra y actualiza esta misma información. El gran problema es que, cuando el primer proceso vuelve a consultar la tabla, se encuentra con valores completamente distintos a los que tenía al principio. Como su nombre lo indica, no puedes repetir la lectura original y obtener lo mismo porque alguien más ya modificó el escenario.

Ejemplo:

Proceso A:

```
--- PROCESO A: LECTURA NO REPETIBLE -----  
USE AdventureWorks2025;  
GO  
  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
DECLARE @Count int;  
DECLARE @Total money;  
DECLARE @Average money;  
  
PRINT 'Proceso A: 1. Calculando el total de órdenes...';  
SELECT @Count = COUNT(DISTINCT SalesOrderID)  
FROM Sales.SalesOrderDetail;  
  
PRINT 'Proceso A: Esperando 5 segundos (dando tiempo al Proceso B para su primer UPDATE)...';  
WAITFOR DELAY '00:00:5';  
  
PRINT 'Proceso A: 3. Calculando la suma total...';  
SELECT @Total = SUM(UnitPrice * OrderQty * (1 - UnitPriceDiscount))  
FROM Sales.SalesOrderDetail;  
  
PRINT 'Proceso A: Esperando 5 segundos (dando tiempo al Proceso B para su segundo UPDATE)...';  
WAITFOR DELAY '00:00:5';  
  
PRINT 'Proceso A: 5. Calculando el promedio general...';  
SELECT @Average = AVG(TotalPrice)  
FROM (  
    SELECT SalesOrderID, SUM(UnitPrice * OrderQty * (1 - UnitPriceDiscount)) AS TotalPrice  
    FROM Sales.SalesOrderDetail  
    GROUP BY SalesOrderID  
) AS TotOrders;  
  
PRINT 'Proceso A: Mostrando la discrepancia de resultados...';  
PRINT '    Actual AVR: ' + CAST(@Average AS VARCHAR(20))  
    + '    Calc AVR: ' + CAST(@Total / @Count AS VARCHAR(20));  
GO
```

Proceso B:

```

--- PROCESO B: LECTURA NO REPETIBLE -----
USE AdventureWorks2025;
GO

PRINT 'Proceso B: Esperando 5 segundos para que A ejecute su primer cálculo...';
WAITFOR DELAY '00:00:05';

PRINT 'Proceso B: 2. Ejecutando el primer UPDATE (Modificando cantidades)...';
UPDATE Sales.SalesOrderDetail
SET OrderQty = 600
WHERE SalesOrderID = 43659 AND ProductID = 776;

PRINT 'Proceso B: Esperando 5 segundos para que A ejecute su segundo cálculo...';
WAITFOR DELAY '00:00:5';

PRINT 'Proceso B: 4. Ejecutando el segundo UPDATE (Modificando descuentos)...';
UPDATE Sales.SalesOrderDetail
SET UnitPriceDiscount = 0.4
WHERE ProductID = 776;

PRINT 'Proceso B: Actualizaciones terminadas.';
GO
-- Revertir cambios -----
UPDATE Sales.SalesOrderDetail
SET OrderQty = 1
WHERE SalesOrderID = 43659 AND ProductID = 776;
GO
UPDATE Sales.SalesOrderDetail
SET UnitPriceDiscount = 0
WHERE ProductID = 776;

```

Resultado de Proceso A:

```

Proceso A: 1. Calculando el total de órdenes...
Proceso A: Esperando 5 segundos (dando tiempo al Proceso B para su primer UPDATE)...
Proceso A: 3. Calculando la suma total...
Proceso A: Esperando 5 segundos (dando tiempo al Proceso B para su segundo UPDATE)...
Proceso A: 5. Calculando el promedio general...
Proceso A: Mostrando la discrepancia de resultados...
    Actual AVR:3530.19    Calc AVR: 3491.64

Hora de finalización: 2026-09-22T15:45:56.2292449-06:00

```

Resultado de Proceso B:

```

Proceso B: Esperando 5 segundos para que A ejecute su primer cálculo...
Proceso B: 2. Ejecutando el primer UPDATE (Modificando cantidades)...

(1 fila afectada)
Proceso B: Esperando 5 segundos para que A ejecute su segundo cálculo...
Proceso B: 4. Ejecutando el segundo UPDATE (Modificando descuentos)...

(228 filas afectadas)
Proceso B: Actualizaciones terminadas.

Hora de finalización: 2026-09-22T15:45:59.1066700-06:00

```

Conclusión:

Al finalizar esta prueba, obtuvimos un choque de realidades al final del experimento. Cuando el Proceso A intenta sacar su promedio general, los resultados muestran una clara discrepancia porque sus cálculos se hicieron sobre datos que estaban cambiando a sus espaldas. El sistema nos arroja que el promedio actual real quedó en 3530.19, pero el cálculo matemático que arrastraba nuestro proceso terminó en 3491.64. La lección que nos deja esto es que, si estuviéramos diseñando algo que requiera alta precisión, como un sistema de monitoreo de sensores o un panel de control para redes, terminaríamos tomando decisiones basadas en métricas e informes totalmente descuadrados.

LECTURA FANTASMA

Una lectura fantasma ocurre cuando estás consultando un grupo de registros bajo ciertas condiciones y, de repente, en una segunda lectura idéntica dentro de la misma transacción, aparecen datos nuevos que antes no estaban. Esto pasa porque otro proceso diferente insertó información nueva que ahora cumple con tu búsqueda original. A diferencia de la lectura no replicable donde los valores de los datos cambiaban, aquí lo que cambia es la cantidad de filas porque alguien más agregó registros a la tabla a tus espaldas.

Ejemplo:

Proceso A:

```
--PROCESO A: LECTURA FANTASMA -----
USE AdventureWorks2025; -- Ajusta según tu versión
GO

-- REPEATABLE READ previene lecturas sucias y no repetibles, pero PERMITE lecturas fantasmas
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

PRINT 'Proceso A: 1. Realizando la primera lectura...';
-- Equivale al primer SELECT de la imagen
SELECT SalesOrderID, ProductID, UnitPrice
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43659;

PRINT 'Proceso A: Esperando 10 segundos para que el Proceso B inserte un nuevo registro...';
WAITFOR DELAY '00:00:10';

PRINT 'Proceso A: 3. Realizando la segunda lectura (Aparecerá el registro fantasma)...';
-- Equivale al segundo SELECT de la imagen
SELECT SalesOrderID, ProductID, UnitPrice
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43659;

COMMIT TRANSACTION;
PRINT 'Proceso A: Transacción finalizada.';
GO
```

Proceso B:

```
--PROCESO B: LECTURA FANTASMA -----
USE AdventureWorks2025;
GO

PRINT 'Proceso B: Esperando 5 segundos antes de insertar...';
WAITFOR DELAY '00:00:05';

PRINT 'Proceso B: 2. Insertando un nuevo registro (El fantasma)...';

-- Equivale al INSERT de la imagen.
-- Insertamos un nuevo detalle para la misma orden (43659) con un producto diferente (ej. 707)
INSERT INTO Sales.SalesOrderDetail
(SalesOrderID, OrderQty, ProductID, SpecialOfferID, UnitPrice, UnitPriceDiscount)
VALUES
(43659, 10, 707, 1, 35.00, 0.0);

PRINT 'Proceso B: Inserción terminada.';
GO

-- RESTAURAR DATOS (BORRAR REGISTRO)
DELETE FROM Sales.SalesOrderDetail WHERE SalesOrderID = 43659 AND ProductID = 707;
```

Resultado en Proceso A:

Resultados Mensajes

Proceso A: 1. Realizando la primera lectura...

(12 filas afectadas)

Proceso A: Esperando 10 segundos para que el Proceso B inserte un nuevo registro...

Proceso A: 3. Realizando la segunda lectura (Aparecerá el registro fantasma)...

(13 filas afectadas)

Proceso A: Transacción finalizada.

Hora de finalización: 2026-09-22T15:51:24.0201219-06:00

|

Resultados

Mensajes

	SalesOrderID	ProductID	UnitPrice
1	43659	776	2024.994
2	43659	777	2024.994
3	43659	778	2024.994
4	43659	771	2039.994
5	43659	772	2039.994
6	43659	773	2039.994
7	43659	774	2039.994
8	43659	714	28.8404
9	43659	716	28.8404
10	43659	709	5.70
11	43659	712	5.1865
12	43659	711	20.1865

	SalesOrderID	ProductID	UnitPrice
1	43659	776	2024.994
2	43659	777	2024.994
3	43659	778	2024.994
4	43659	771	2039.994
5	43659	772	2039.994
6	43659	773	2039.994
7	43659	774	2039.994
8	43659	714	28.8404
9	43659	716	28.8404
10	43659	709	5.70
11	43659	712	5.1865
12	43659	711	20.1865
13	43659	707	35.00

Resultado en Proceso B:

```
Mensajes
Proceso B: Esperando 5 segundos antes de insertar...
Proceso B: 2. Insertando un nuevo registro (El fantasma)...

(1 fila afectada)
Proceso B: Inserción terminada.

Hora de finalización: 2026-09-22T15:51:21.4674003-06:00
```

Conclusión:

Los resultados de la imagen muestran cómo se materializa este error. En la primera lectura, el sistema nos indica que encontró un total de 12 filas afectadas. En la tabla de la izquierda vemos perfectamente esos doce registros listados. Sin embargo, después de la pausa de diez segundos para darle oportunidad al Proceso B de hacer la inserción, el Proceso A vuelve a ejecutar exactamente la misma consulta y ahora nos dice que hay 13 filas afectadas. Al observar la tabla de la derecha, vemos claramente que apareció un registro extra en la posición trece, con un producto diferente y un precio unitario de 35.00.

Lo que obtuvimos es la demostración perfecta de que, aunque nuestro proceso original bloqueó los registros existentes para que no los modificaran, fue incapaz de evitar que se colara un dato extra.

REFERENCIAS

1. Guerrero, F. G., & Rojas, C. E. (2001). **Microsoft SQL Server 2000 Programming by Example**. Quest Custom Publishing.
2. Özsu, M. T., & Valduriez, P. (2020). **Principles of Distributed Database Systems (4.^a ed.)**. Springer.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). **Fundamentos de Bases de Datos (7.^a ed.)**. McGraw-Hill.